Lab Assignment 3: Practicing SQL queries using Operators, Aggregate and Scalar functions

Objective: To solve simple queries using SQL operators, Aggregate functions and Scalar functions over tables based on assignment 1.

1. Write the SQL Expressions for the following queries using suitable SQL operators.

a) Display the account numbers, account types, and balances of accounts that belong to any of the following branch codes: B001, B002, B003, and B005.

```
mysql> SELECT ACC_NO, ACC_TYPE, BALANCE
    -> FROM ACCOUNT
    -> WHERE BRANCH_CODE IN ('B001','B002','B003','B005');
+--------+----------+---------+
| ACC_NO | ACC_TYPE | BALANCE |
+--------+----------+---------+
| A1001  | Savings  |   25000 |
| A1002  | Current  |   40000 |
| A1003  | Savings  |   18000 |
| A1005  | Current  |   50000 |
+--------+----------+---------+
4 rows in set (0.00 sec)
```

b) Display the records of customers whose names start with the letter 'A' and who live in the city of Delhi.

```
mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE CUST_NAME LIKE 'A%'
    -> AND ADDRESS='Delhi';
+---------+-------------+---------+------------+
| CUST_NO | CUST_NAME   | ADDRESS | PHONE      |
+---------+-------------+---------+------------+
| C001    | Ankit Kumar | Delhi   | 9876543210 |
+---------+-------------+---------+------------+
1 row in set (0.00 sec)
```

c) Display customer number, account number, original balance, interest amount, and final balance by calculating 5% interest on the account balance.

```
mysql> SELECT CUST_NO, ACC_NO, BALANCE,
    ->        BALANCE*0.05 AS INTEREST,
    ->        BALANCE + BALANCE*0.05 AS FINAL_BALANCE
    -> FROM ACCOUNT;
+---------+--------+---------+----------+---------------+
| CUST_NO | ACC_NO | BALANCE | INTEREST | FINAL_BALANCE |
+---------+--------+---------+----------+---------------+
| C001    | A1001  |   25000 |  1250.00 |      26250.00 |
| C002    | A1002  |   40000 |  2000.00 |      42000.00 |
| C003    | A1003  |   18000 |   900.00 |      18900.00 |
| C004    | A1004  |   32000 |  1600.00 |      33600.00 |
| C005    | A1005  |   50000 |  2500.00 |      52500.00 |
+---------+--------+---------+----------+---------------+
5 rows in set (0.00 sec)
```

d) Display the records of accounts having balance in the range 20000 to 50000.

```
mysql> SELECT *
    -> FROM ACCOUNT
    -> WHERE BALANCE BETWEEN 20000 AND 50000;
+--------+---------+-------------+----------+---------+
| ACC_NO | CUST_NO | BRANCH_CODE | ACC_TYPE | BALANCE |
+--------+---------+-------------+----------+---------+
| A1001  | C001    | B001        | Savings  |   25000 |
| A1002  | C002    | B002        | Current  |   40000 |
| A1004  | C004    | B004        | Savings  |   32000 |
| A1005  | C005    | B005        | Current  |   50000 |
+--------+---------+-------------+----------+---------+
4 rows in set (0.00 sec)
```

e) Display the records of customers having the second letter in their name as ‘m’.

```
mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE CUST_NAME LIKE '_m%';
+---------+------------+---------+------------+
| CUST_NO | CUST_NAME  | ADDRESS | PHONE      |
+---------+------------+---------+------------+
| C005    | Amit Singh | Lucknow | 8899776655 |
+---------+------------+---------+------------+
1 row in set (0.00 sec)
```

f) Display the account numbers of Savings accounts, sorted in descending order of their account balance.

```
mysql> SELECT ACC_NO
    -> FROM ACCOUNT
    -> WHERE ACC_TYPE='Savings'
    -> ORDER BY BALANCE DESC;
+--------+
| ACC_NO |
+--------+
| A1004  |
| A1001  |
| A1003  |
+--------+
3 rows in set (0.00 sec)
```

g) Update all records of the ACCOUNT table by increasing the account balance by 10%.

```
mysql> UPDATE ACCOUNT
    -> SET BALANCE = BALANCE * 1.10;
Query OK, 5 rows affected (0.01 sec)
Rows matched: 5  Changed: 5  Warnings: 0
```

h) Update the account balance by 5% for Savings account holders whose balance is less than 25000.

```
mysql> UPDATE ACCOUNT
    -> SET BALANCE = BALANCE * 1.05
    -> WHERE ACC_TYPE='Savings'
    -> AND BALANCE < 25000;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

i) Display the annual interest amount for each account by calculating interest at 6% per annum.

```
mysql> SELECT ACC_NO, BALANCE,
    ->        BALANCE*0.06 AS ANNUAL_INTEREST
    -> FROM ACCOUNT;
+--------+---------+-----------------+
| ACC_NO | BALANCE | ANNUAL_INTEREST |
+--------+---------+-----------------+
| A1001  |   27500 |         1650.00 |
| A1002  |   44000 |         2640.00 |
| A1003  |   20790 |         1247.40 |
| A1004  |   35200 |         2112.00 |
| A1005  |   55000 |         3300.00 |
+--------+---------+-----------------+
5 rows in set (0.00 sec)
```

j) Update the account type from ‘Current’ to ‘Savings’ for accounts having balance less than 45000.

```
mysql> UPDATE ACCOUNT
    -> SET ACC_TYPE='Savings'
    -> WHERE ACC_TYPE='Current'
    -> AND BALANCE < 45000;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

k) Delete the record of the customer having loan amount less than 150000.

```
mysql> DELETE FROM CUSTOMER
    -> WHERE CUST_NO IN (
    ->     SELECT CUST_NO
    ->     FROM LOAN
    ->     WHERE AMOUNT < 150000
    -> );
Query OK, 1 row affected (0.00 sec)
```

l) Delete the customer records whose names end with the letter 'h'.

```
mysql> DELETE FROM CUSTOMER
    -> WHERE CUST_NAME LIKE '%h';
Query OK, 1 row affected (0.00 sec)
```

2. Write the SQL Expressions for the following queries using suitable SQL aggregate function.

a) Display the average account balance of accounts with the account type 'Savings'.

```
mysql> SELECT AVG(BALANCE)
    -> FROM ACCOUNT
    -> WHERE ACC_TYPE='Savings';
+--------------+
| AVG(BALANCE) |
+--------------+
|   31872.5000 |
+--------------+
1 row in set (0.01 sec)
```

b) Display the transaction type and the average transaction amount of transactions performed by users.

```
mysql> SELECT TRANS_TYPE, AVG(TRANS_AMOUNT)
    -> FROM TRANSACTION_DETAILS
    -> GROUP BY TRANS_TYPE;
+------------+-------------------+
| TRANS_TYPE | AVG(TRANS_AMOUNT) |
+------------+-------------------+
| Deposit    |         7333.3333 |
| Withdraw   |         2500.0000 |
+------------+-------------------+
2 rows in set (0.00 sec)
```

c) Display the account number, customer number, and balance of the account having the highest balance.

```
mysql> SELECT ACC_NO, CUST_NO, BALANCE
    -> FROM ACCOUNT
    -> WHERE BALANCE = (SELECT MAX(BALANCE) FROM ACCOUNT);
+--------+---------+---------+
| ACC_NO | CUST_NO | BALANCE |
+--------+---------+---------+
| A1005  | C005    |   55000 |
+--------+---------+---------+
1 row in set (0.00 sec)
```

d) Display the total number of customers belonging to the cities Delhi or Mumbai.

```
mysql> SELECT COUNT(*)
    -> FROM CUSTOMER
    -> WHERE ADDRESS IN ('Delhi','Mumbai');
+----------+
| COUNT(*) |
+----------+
|        2 |
+----------+
1 row in set (0.00 sec)
```

e) Display the total loan amount taken by customers from branch code B001, B002 and B004.

```
mysql> SELECT SUM(AMOUNT)
    -> FROM LOAN
    -> WHERE BRANCH_CODE IN ('B001','B002','B004');
+-------------+
| SUM(AMOUNT) |
+-------------+
|      530000 |
+-------------+
1 row in set (0.00 sec)
```

f) Display the account type, the number of accounts, and the total account balance for each account type.

```
mysql> SELECT ACC_TYPE, COUNT(*), SUM(BALANCE)
    -> FROM ACCOUNT
    -> GROUP BY ACC_TYPE;
+----------+----------+--------------+
| ACC_TYPE | COUNT(*) | SUM(BALANCE) |
+----------+----------+--------------+
| Savings  |        4 |       127490 |
| Current  |        1 |        55000 |
+----------+----------+--------------+
2 rows in set (0.00 sec)
```

g) Display the transaction type and the total transaction amount for each transaction type,excluding the account number ‘A1005’.

```
mysql> SELECT TRANS_TYPE, SUM(TRANS_AMOUNT)
    -> FROM TRANSACTION_DETAILS
    -> WHERE ACC_NO <> 'A1005'
    -> GROUP BY TRANS_TYPE;
+------------+------------------+
| TRANS_TYPE | SUM(TRANS_AMOUNT) |
+------------+------------------+
| Deposit    |            12000 |
| Withdraw   |             5000 |
+------------+------------------+
2 rows in set (0.00 sec)
```

h) Display the total account balance of all accounts with the account type ‘Current’.

```
mysql> SELECT SUM(BALANCE)
    -> FROM ACCOUNT
    -> WHERE ACC_TYPE='Current';
+--------------+
| SUM(BALANCE) |
+--------------+
|        55000 |
+--------------+
1 row in set (0.00 sec)
```

i) Display the number of accounts for each account type sorted in increasing order.

```
mysql> SELECT ACC_TYPE, COUNT(*)
    -> FROM ACCOUNT
    -> GROUP BY ACC_TYPE
    -> ORDER BY COUNT(*) ASC;
+----------+----------+
| ACC_TYPE | COUNT(*) |
+----------+----------+
| Current  |        1 |
| Savings  |        4 |
+----------+----------+
2 rows in set (0.00 sec)
```

j) Display the number of transactions for each transaction type in descending order.

```
mysql> SELECT TRANS_TYPE, COUNT(*)
    -> FROM TRANSACTION_DETAILS
    -> GROUP BY TRANS_TYPE
    -> ORDER BY COUNT(*) DESC;
+------------+----------+
| TRANS_TYPE | COUNT(*) |
+------------+----------+
| Deposit    |        3 |
| Withdraw   |        2 |
+------------+----------+
2 rows in set (0.00 sec)
```

k) Display the account type and average account balance, but only for account types whose average balance is greater than 25000.

```
mysql> SELECT ACC_TYPE, AVG(BALANCE)
    -> FROM ACCOUNT
    -> GROUP BY ACC_TYPE
    -> HAVING AVG(BALANCE) > 25000;
+----------+--------------+
| ACC_TYPE | AVG(BALANCE) |
+----------+--------------+
| Savings  |   31872.5000 |
| Current  |   55000.0000 |
+----------+--------------+
2 rows in set (0.00 sec)
```

l) Display the account type and number of accounts, but only for account types having more than two accounts.

```
mysql> SELECT ACC_TYPE, COUNT(*)
    -> FROM ACCOUNT
    -> GROUP BY ACC_TYPE
    -> HAVING COUNT(*) > 2;
+----------+----------+
| ACC_TYPE | COUNT(*) |
+----------+----------+
| Savings  |        4 |
+----------+----------+
1 row in set (0.00 sec)
```

m) Display the account type and maximum account balance for each account type, but only for account types where the maximum balance is greater than 40000.

```
mysql> SELECT ACC_TYPE, MAX(BALANCE)
    -> FROM ACCOUNT
    -> GROUP BY ACC_TYPE
    -> HAVING MAX(BALANCE) > 40000;
+----------+--------------+
| ACC_TYPE | MAX(BALANCE) |
+----------+--------------+
| Savings  |        44000 |
| Current  |        55000 |
+----------+--------------+
2 rows in set (0.00 sec)
```

n) Display the highest account balance among Savings accounts.

```
mysql> SELECT MAX(BALANCE)
    -> FROM ACCOUNT
    -> WHERE ACC_TYPE='Savings';
+--------------+
| MAX(BALANCE) |
+--------------+
|        44000 |
+--------------+
1 row in set (0.00 sec)
```

o) Display the transaction type having the highest total transaction amount. (Optional)

```
mysql> SELECT TRANS_TYPE
    -> FROM TRANSACTION_DETAILS
    -> GROUP BY TRANS_TYPE
    -> ORDER BY SUM(TRANS_AMOUNT) DESC
    -> LIMIT 1;
+------------+
| TRANS_TYPE |
+------------+
| Deposit    |
+------------+
1 row in set (0.00 sec)
```

p) Display the account types whose total account balance is greater than the average account balance of all accounts. (Optional)

```
mysql> SELECT ACC_TYPE
    -> FROM ACCOUNT
    -> GROUP BY ACC_TYPE
    -> HAVING SUM(BALANCE) >
    -> (
    ->     SELECT AVG(BALANCE)
    ->     FROM ACCOUNT
    -> );
+----------+
| ACC_TYPE |
+----------+
| Savings  |
| Current  |
+----------+
2 rows in set (0.00 sec)
```

3. Write the SQL Expressions for the following queries using suitable SQL scalar function.

a) Display 2nd- 6th characters of your name.

```
mysql> SELECT SUBSTRING('Koushik',2,5);
+--------------------------+
| SUBSTRING('Koushik',2,5) |
+--------------------------+
| oushi                    |
+--------------------------+
1 row in set (0.00 sec)
```

b) Find length of your full university name.

```
mysql> SELECT LENGTH('Siksha O Anusandhan University');
+-----------------------------------------+
| LENGTH('Siksha O Anusandhan University') |
+-----------------------------------------+
|                                      30 |
+-----------------------------------------+
1 row in set (0.00 sec)
```

c) Display all the customer names in upper case.

```
mysql> SELECT UPPER(CUST_NAME)
    -> FROM CUSTOMER;
+------------------+
| UPPER(CUST_NAME) |
+------------------+
| ANKIT KUMAR      |
| ANJALI VERMA     |
| NEHA SHARMA      |
+------------------+
3 rows in set (0.00 sec)
```

d) Display the account type of each customer as a two letter code.

```
mysql> SELECT LEFT(ACC_TYPE,2)
    -> FROM ACCOUNT;
+------------------+
| LEFT(ACC_TYPE,2) |
+------------------+
| Sa               |
| Sa               |
| Sa               |
| Sa               |
| Cu               |
+------------------+
5 rows in set (0.00 sec)
```

e) Display the month and year of the transaction perform by each customer.

```
mysql> SELECT MONTH(TRANS_DATE), YEAR(TRANS_DATE)
    -> FROM TRANSACTION_DETAILS;
+-------------------+------------------+
| MONTH(TRANS_DATE) | YEAR(TRANS_DATE) |
+-------------------+------------------+
|                 1 |             2025 |
|                 1 |             2025 |
|                10 |             2025 |
|                 1 |             2024 |
|                 3 |             2024 |
+-------------------+------------------+
5 rows in set (0.00 sec)
```

f) Display the transaction date of each customer in dd/mm/yyyyy format.

```
mysql> SELECT DATE_FORMAT(TRANS_DATE,'%d/%m/%Y')
    -> FROM TRANSACTION_DETAILS;
+-----------------------------------+
| DATE_FORMAT(TRANS_DATE,'%d/%m/%Y') |
+-----------------------------------+
| 10/01/2025                        |
| 12/01/2025                        |
| 15/10/2025                        |
| 18/01/2024                        |
| 20/03/2024                        |
+-----------------------------------+
5 rows in set (0.00 sec)
```

g) To calculate the exact age of each transaction relative to the current date in term of years,months and days.

```
mysql> SELECT DATEDIFF(CURDATE(), TRANS_DATE)
    -> FROM TRANSACTION_DETAILS;
+---------------------------------+
| DATEDIFF(CURDATE(), TRANS_DATE) |
+---------------------------------+
|                             433 |
|                             431 |
|                             155 |
|                             791 |
|                             729 |
+---------------------------------+
5 rows in set (0.00 sec)
```

h) Display the date corresponding to 15 days after today's date.

```
mysql> SELECT DATE_ADD(CURDATE(), INTERVAL 15 DAY);
+--------------------------------------+
| DATE_ADD(CURDATE(), INTERVAL 15 DAY) |
+--------------------------------------+
| 2026-04-03                           |
+--------------------------------------+
1 row in set (0.01 sec)
```

i) Display the value 94204.27348 truncated up to 2 digits after decimal point.

```
mysql> SELECT TRUNCATE(94204.27348,2);
+-------------------------+
| TRUNCATE(94204.27348,2) |
+-------------------------+
|                94204.27 |
+-------------------------+
1 row in set (0.00 sec)
```

j) Display the value of the expression 5 + 89.

```
mysql> SELECT 5 + POWER(8,9);
+----------------+
| 5 + POWER(8,9) |
+----------------+
|      134217733 |
+----------------+
1 row in set (0.00 sec)
```

k) Find out the square root of 6464312.

```
mysql> SELECT SQRT(6464312);
+--------------------+
| SQRT(6464312)      |
+--------------------+
| 2542.501130776543 |
+--------------------+
1 row in set (0.00 sec)
```

l) Display the string “HELLO ITER” in lower case with a column heading lower case.

```
mysql> SELECT LOWER('HELLO ITER') AS LOWER_CASE;
+------------+
| LOWER_CASE |
+------------+
| hello iter |
+------------+
1 row in set (0.00 sec)
```